Lifecycle init-ordering bug-class audit — ApiKeyProvider sibling hunt

Date: 2026-06-14 **Trigger:** Severe production bug in `api-app/.../handoff/ApiKeyProvider.kt` — its `onCreate()` cached the key/port lambdas gated on `ApiApplication.controllerHolder/ keyStoreHolder != null`, on the FALSE comment "ContentProvider.onCreate() runs AFTER Application.onCreate()". Android runs `ContentProvider.onCreate()` BEFORE `Application.onCreate()`, so the holders (populated in `ApiApplication.onCreate()`) were always null → cache stayed `{ null }` → empty cursor forever → every auth-gated `/v1/{provider}` search 401'd. (Already fixed at source — see verdict #1.)

Scope: all `ContentProvider` subclasses, companion/global holders handed off across components, and once-snapshot-from-later-mutable-state patterns, across `api-app` + client `app`.

Documented Android init order (the contract every finding is measured against)

```
Application.attachBaseContext(base)
  → [each <provider>] ContentProvider.onCreate()      ← runs HERE,
FIRST
Application.onCreate()                                  ← runs
AFTER all providers
  → Activity / Service lifecycles
```

Source: Android `ActivityThread.handleBindApplication` calls `installContentProviders()` (which invokes each provider's `onCreate()`) BEFORE `mInstrumentation.callApplicationOnCreate()`. Consequence: any `ContentProvider.onCreate()` that reads state written in `Application.onCreate()` reads it as still-uninitialised.

Candidates audited

1. `api-app/.../handoff/ApiKeyProvider.kt` — the original bug (FIXED)

- **Holders read:** `ApiApplication.controllerHolder` (:83), `keyStoreHolder` (:84), `controllerHolder?.state?.value` (:90) — all populated in `ApiApplication.onCreate()` (:49-50).
- **Init order:** provider `onCreate()` (:57) runs BEFORE `ApiApplication.onCreate()`.

- **Verdict:** was BUG (cache-in-onCreate of later-populated holders); now **SAFE/FIXED.** The production default lambdas (:46 `keyProvider`, :53 `portProvider`) resolve the holders + the `Running` state LAZILY per `query()` via `resolveRunningKey()` (:82) / `resolveRunningPort()` (:89). `onCreate()` (:57-78) now caches nothing and just `return true`. Because `state` is a live `MutableStateFlow` (`ApiEngineController.kt:44–45`) read by `.value` on every query, it is also clause-3 safe (no stale snapshot).
- **Fix pattern (canonical):** resolve lazily per-use; never cache in `onCreate` from later-populated state.

2. `app/.../handoff/ApiKeyClient.kt` — client-side reader

- Queries the provider lazily on every `read()` call (:37-72) via `context.contentResolver.query(...)`. Holds NO companion state; the only companion member is a `const TAG` (:74-78).
- **Init order:** N/A — no early caching; resolves at call time.
- **Verdict: SAFE.**

3. `ApiApplication` companion holders (`controllerHolder`, `keyStoreHolder`)

- `@Volatile var ... = null`, `private set`, set only in `onCreate()` (:49-50) + `@VisibleForTesting setHoldersForTest` (:101).
- **Only reader in the entire repo:** `ApiKeyProvider` (verified by repo-wide grep — no other `*.kt` reads either holder). Since the sole reader now reads lazily, the hand-off has no remaining null-at-read window in production.
- **Verdict: SAFE** (given finding #1's fix).

4. `app/.../MainActivity.kt` — `theme` / `showOnboarding`

- `var theme/showOnboarding by mutableStateOf(null)` are LOCAL vars inside `onCreate()` (:101-102), collected live from flows in two `repeatOnLifecycle` blocks (:119-129). They are not companion/global holders and not read by any earlier component.
- **Verdict: SAFE** (live flow collection, not an early snapshot).

5. `app/.../crash/NavTeardownCrashReporter.kt`

- `install()` (:69) reads `Thread.getDefaultUncaughtExceptionHandler()` at install time and chains it; `uncaughtException` resolves `previous` at crash time. No Application-scoped state cached early. Companion holds only constants (:52-61).
- **Verdict: SAFE.**

6. `app/.../LavaApplication.kt`

- No companion holders published for earlier components; companion holds only `TAG` (:108). Reads its own `@Inject` deps inside `onCreate()` (correct — Hilt-injected by then).
- **Verdict: SAFE.**

7. api-app `MainActivity` / `ApiEngineService` `onCreate()`

- Neither reads `controllerHolder`/`keyStoreHolder` (grep-verified). Activities/Services run AFTER `Application.onCreate()` regardless, so even if they did it would be ordered correctly.
- **Verdict: SAFE.**

Negative-space checks (patterns deliberately searched for, none found)

- **AndroidX App Startup `Initializer<>`:** none in `api-app/app` (grep `: Initializer<` empty). No `androidx.startup.InitializationProvider` in either manifest.
- **`attachBaseContext` overrides:** none (grep empty).
- **`by lazy` snapshots of holder/engine/state in main source:** none (grep empty).
- **Other `ContentProvider` subclasses:** only `ApiKeyProvider` (production) + `FileProvider` (framework, no custom caching) + the test `FakeKeyProvider` stub. The app manifest `<provider>` is just `androidx.core.content.FileProvider`.
- **Other companion `var ...Holder` published across components:** only the two `ApiApplication` holders (grep-verified).

Test-bluff note (why the original bug shipped green)

`app/src/test/.../ApiKeyClientTest.kt` is a CLIENT-side test that registers a `FakeKeyProvider` stub (:93) via `ShadowContentResolver` and exercises `ApiKeyClient.read()`. It NEVER instantiates the real `ApiKeyProvider`, never drives its `onCreate()`, and never touches `ApiApplication`'s holder hand-off. So the failing component's real lifecycle path (`ApiKeyProvider.onCreate()` → holder read) was entirely outside any test's reach — the canonical §6.J bluff: the test that "covers the handoff" structurally cannot fail when the provider half of the handoff breaks. A real-stack test must instantiate `ApiKeyProvider`, populate holders only AFTER `attachInfo`/onCreate (simulating real ordering), and assert `query()` returns the row once the engine reaches `Running` — i.e. assert the LAZY resolution, not a `withFakes`-substituted lambda. (Recommended; not implemented in this read-only audit.)

Bottom line

`ApiKeyProvider` is an **ISOLATED case** of the inverted-lifecycle / cache-in-onCreate bug class. Repo-wide there is exactly ONE cross-component companion hand-off (`ApiApplication` holders → `ApiKeyProvider`), and after the source fix its sole reader resolves lazily, closing the null-at-read window. No sibling ContentProvider, App Startup Initializer, `attachBaseContext`, or `by lazy` snapshot reads Application-scoped state early. **No further lazy-resolution fixes are required.** The remaining gap is a test gap, not a code gap: the broken path had no real-stack test (item above).